

Table des matières

Préambule	1
Documentation	1
Introduction.....	2
Figures	3
OperationFigure.....	4
Modèle de dessin	5
Les Outils (package tools)	6
Interface Graphique avec JavaFX.....	6
Vue	6
Des menus.....	7
Des ContextMenu.....	8
Des widgets customizables	8
Contrôleur	8
Travail à réaliser.....	11
Planning.....	12
Mise en place et utilisation de JavaFX	12
SceneBuilder.....	13
Mise en place de JavaFX dans VSCode.....	13
Mise en place de JavaFX dans Eclipse.....	14

Préambule

Vous pourrez trouver une archive contenant l'ébauche du code à réaliser dans
/pub/FISE_LAOB12/TPs/TP4/

Sauf mention explicite, la notation utilisée ici sera la notation Java : e.g. "public int size();" plutôt que
UML : e.g. "+size() : int".

Documentation

- Documentation Java 21 sur Oracle.com : <https://docs.oracle.com/en/java/javase/21/docs/api/>
- Documentation Java 21 locale : file:///pub/FISE_LAOB12/docs/index.html
- Documentation JavaFX : <https://openjfx.io/javadoc/20/>

Introduction

Le but de ce TP qui s'étalera sur l'ensemble des séances restantes est de vous faire travailler sur une véritable application : Un mini éditeur de formes géométriques.

L'éditeur est composé :

- D'un modèle de dessin gérant le dessin de plusieurs formes géométriques (Cercle, Ellipse, Rectangle).
- D'une interface graphique réalisée avec JavaFX permettant :
 - De commander les paramètres du modèle de dessin (type de forme, couleurs de trait et de remplissage, type de trait (continu, pointillé, sans trait) et épaisseur du trait.
 - De dessiner les formes gérées par le modèle de dessin à la souris.
 - De sélectionner des figures puis d'éditer les figures sélectionnées et de réaliser des :
 - Déplacement, rotation, facteur d'échelle.
 - Changements de style.
 - Réordonnancement des figures.
 - Destructions des figures sélectionnées.
 - Opérations booléennes entre deux figures :
 - Intersection
 - Soustraction
 - Union
 - D'afficher dans un panneau d'informations les informations relatives à la figure située sous le curseur de la souris dans la zone de dessin.
 - De gérer les Undos / Redos à chaque fois qu'une figure est ajoutée, supprimée ou modifiée.
 - D'effacer l'ensemble des figures.

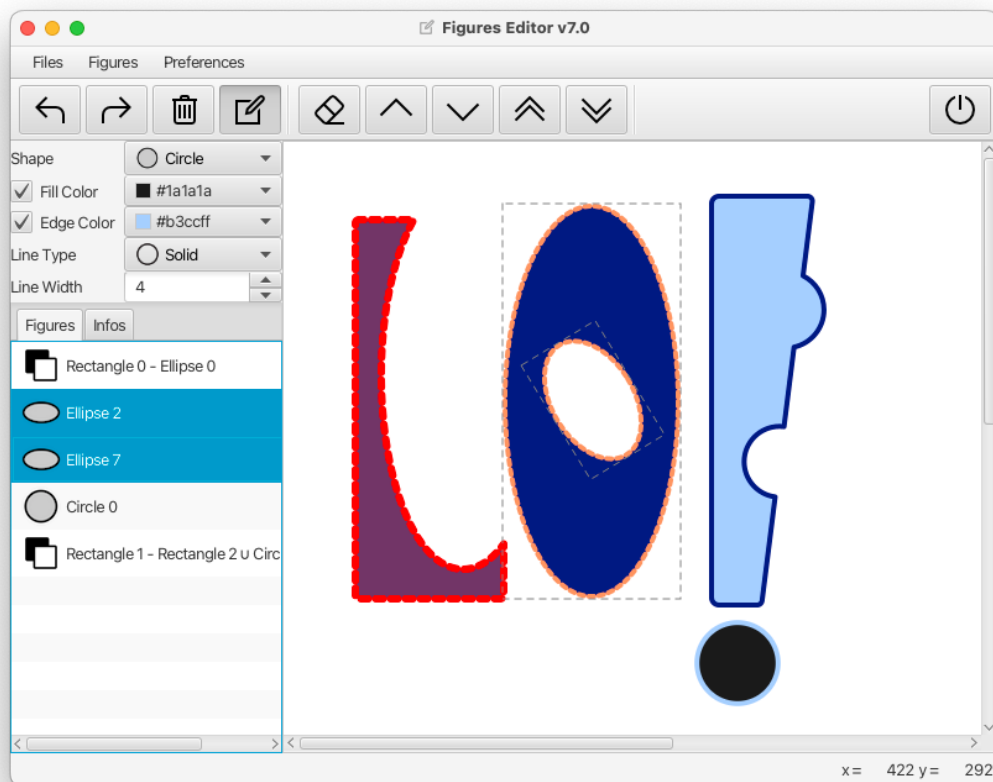


Figure 1 : Éditeur de formes géométriques

Figures

La classe Figure qui vous est fournie sera la classe mère de toutes les figures que vous aurez à construire. Les figures sont composées (entre autres) de :

- **Shape** shape : une forme géométrique à dessiner (au sens de JavaFX). Cette forme est la représentation géométrique de notre figure dans la zone de dessin (un simple Panel).
- **Color** edge/fill : les couleurs de remplissage et de trait des figures à dessiner, une figure peut ne pas avoir de couleur remplissage ou de couleur de trait, mais pas les deux en même temps. L'une ou l'autre doit subsister.
- **LineType** : le style du trait (continu, pointillé ou sans trait).
- Une épaisseur de trait (comprise entre 1 et 32).
- Un nom (le type de la figure) et un numéro d'instance unique pour chaque figure du même type qui permettra de distinguer les figures d'un même type entre elles et d'afficher leur nom dans la liste des figures ainsi que dans un panneau d'information dans l'interface graphique.
- **Rectangle** selectionRectangle : un rectangle JavaFX qui permettra de représenter la sélection / désélection de cette figure dans la zone de dessin.
- **Group** root : un groupe JavaFX contenant shape et éventuellement selectionRectangle afin que les deux restent toujours liées même si la figure subit des transformations géométriques (translation, rotation, facteur d'échelle). C'est donc sur ce groupe que s'appliqueront ces transformations.
- Un état de sélection indiquant si la figure est sélectionnée ou non. Lorsque la figure est sélectionnée, selectionRectangle est ajouté à root et inversement lorsque la figure est désélectionnée.
- Puisque JavaFX implémente un graphe de scène, dessiner une figure dans la zone de dessin consiste simplement à ajouter le root d'une figure aux enfants de la zone de dessin. Et effacer une figure consiste à retirer le root d'une figure de la zone de dessin.

➤ Vous devrez créer les classes filles suivantes : Circle, Rectangle, OperationFigure.

Toutes les classes représentant des figures que l'on doit pouvoir dessiner à la souris dans la zone de dessin (Ellipse, Circle, Rectangle), implémentent l'interface Mouseable qui permet de créer une figure à un point précis (x, y) en cliquant, puis d'augmenter la taille de la figure en cours de construction en tirant la souris (toujours avec le bouton de la souris enfoncé) et enfin de relâcher le bouton de la souris pour terminer la figure.

Les figures contiennent les méthodes nécessaires pour :

- Créer une nouvelle figure à un point donné de taille 0 : Ce constructeur sera utilisé pour créer une nouvelle figure à partir d'évènements souris.
- Créer une nouvelle figure à un point donné avec l'ensemble de ses caractéristiques : Par exemple le rayon pour un cercle, la largeur et la hauteur pour une ellipse ou un rectangle.
- Déplacer le dernier point de la figure (ce qui permet de créer des figures de tailles non nulles à la souris). Cette méthode pourra être assortie de toutes les méthodes nécessaires à la création d'une nouvelle figure à l'aide d'évènements souris.
- Créer une figure par copie afin de créer une figure identique mais distincte. Ce constructeur de copie pourra être utilisé pour cloner les figures.
- Obtenir (et/ou) de modifier les attributs mentionnés plus haut.
- Obtenir :
 - Le nom de la figure constitué du nom de la classe suivi par un numéro d'instance unique dans chaque type de figure.
 - Le centre de la figure.
 - La largeur de la figure.
 - La hauteur de la figure.

- Le point en haut à gauche et le point en bas à droite.
- Ces méthodes pourront être utilisées pour déterminer le `selectionRectangle` et/ou pour remplir un panneau d'information lorsque la souris passera au-dessus d'une figure dans la zone de dessin.
- De quoi cloner une figure grâce à une méthode public `Figure clone()` définie dans une interface `Prototype<Figure>` qui étends l'interface `Cloneable`. Vous noterez que la méthode `clone` de la classe `Object` était quant à elle protégée et donc non accessible à l'extérieur de la classe.
- Les opérations classiques de la classe `Object` : `equals`, `hashCode`, `toString`. Une méthode `equals(Figure)` sera implémentée dans chaque sous classe. Cette méthode est indispensable car elle permettra de comparer les figures lors de la création ou de la mise en place des Mementos dans la gestion des Undos / Redos afin de ne pas produire de doublons dans les Mementos.
- Toutes les opérations sur les figures faisant intervenir des événements souris seront déléguées à des outils (rassemblés dans le package `tools`. Voir la section Les Outils (package `tools`).

OperationFigure

La classe `OperationFigure` est une `Figure` particulière permettant de représenter le résultat d'opérations booléennes entre deux autres figures, elle n'implémente donc pas l'interface `Mouseable` car sa création requiert deux autres figures déjà existantes.



Si l'on part des 2 figures suivantes :

On doit pouvoir réaliser les opérations suivantes :



Intersection



Soustraction



Union

La figure résultante de l'opération prendra les attributs de dessin de la première des deux figures : Couleur de trait, couleur de remplissage, type de trait, épaisseur de trait.

La classe `OperationFigure` contient :

- Les deux figures à l'origine de l'opération (afin que l'on puisse reséparer ces deux figures en deux figures distinctes). Ces deux figures seront "finales" dans la mesure où elles ne seront pas modifiées.
- Le type de l'opération à réaliser, représenté ici par une instance de l'enum `OperationType`.
- `public OperationFigure(Figure figure1, Figure figure2, OperationType operationType, Logger parentLogger)` : Un constructeur valué à partir de deux figures, du type d'opération à réaliser et du `Logger` à utiliser pour les messages d'erreur.
 - Lève une `IllegalArgumentException` si
 - `figure1` et `figure2` sont les mêmes.
 - `figure1` et `figure2` n'ont pas d'overlap.
 - Vous pourrez utiliser la méthode `Shape operate(Shape s1, Shape s2)` de l'enum `OperationType` pour réaliser l'opération booléenne requise.

- **Attention** : Afin de pouvoir réaliser ces opérations booléennes sur des figures pouvant avoir subi des transformations (translations, rotations, facteurs d'échelle), il faudra copier les transformations du groupe parent (root) dans la shape elle-même.
- `public OperationFigure(Figure figure) throws IllegalArgumentException` : Un constructeur de copie à partir d'une autre Figure afin de créer une copie distincte mais au contenu identique d'une OperationFigure.
 - On lèvera une `IllegalArgumentException` si figure n'est pas une OperationFigure.
 - **Attention** : la classe Shape n'ayant pas de constructeur de copie, il faudra recréer l'opération booléenne tout comme dans le constructeur précédent.
- `public OperationType getOperationType()` : un accesseur permettant d'obtenir le type d'opération.
- `public List<Figure> getFigures()` : un accesseur permettant d'obtenir les figures internes à l'origine de l'opération dans leur état initial.
 - **Attention** : A l'inverse du constructeur, il faudra retirer les transformations que vous aviez ajoutées à shape dans chaque copie de figure pour les remettre dans le groupe parent (root).
- Des implémentations des méthodes de la classe abstraite Figure :
 - `public Figure clone()` : pour cloner la figure.
 - `public int hashCode()` : pour calculer le hashCode de la figure. Vous pourrez ici utiliser la génération automatique de cette méthode proposée par votre IDE (en se basant simplement sur les 3 attributs : figure1, figure2 et operationType).
 - `protected boolean equals(Figure figure)` : qui renverra vrai si figure est aussi une OperationFigure contenant des figures identiques et possédant un type d'opération identique.
 - `public String toString()` : qui renverra une chaîne de caractères représentant cette figure. Contrairement aux autres types de figures qui utilisent la norme "Nom de la figure" + "numéro d'instance", on utilisera ici la norme "toString de la figure 1" + "toString du type d'opération" + "toString de la figure 2".

Modèle de dessin

Le modèle de dessin représente le cœur de notre application (notre modèle de données) et est implémenté dans la classe Drawing. La classe Drawing se présente comme suit :

- Elle contient une liste de Figures (les figures à dessiner et à gérer).
- Elle se comporte comme une Liste Observable de Figures (en héritant de la classe [ModifiableObservableListBase<Figure>](#)) ce qui lui permettra d'être utilisée comme contenu d'une [ListView<Figure>](#) JavaFX.
- Elle se comporte aussi comme un [Originator<Figure>](#) afin de pouvoir créer un Memento sauvegardant sa liste de figures ou de mettre en place une nouvelle liste de figures à partir d'un Memento qui lui est fourni. Ceci permettra de gérer les Undos / Redos pour toutes les opérations qui créent, suppriment ou modifient les figures de la liste.
- Et enfin elle implémente l'interface [ListChangeListener<Figure>](#) afin qu'elle puisse être notifiée des changements de sélection dans la [ListView<Figure>](#) qui servira à afficher la liste des figures pour que ces changements de sélection puissent se refléter dans la zone de dessin où sont dessinées les figures.
- La classe Drawing gère aussi les paramètres du dessin:
 - Les couleurs de remplissage et de trait à appliquer aux prochaines figures à dessiner.
 - Le type et la largeur du trait.
 - Ces paramètres sont implémentés en utilisant des propriétés JavaFX afin que ces propriétés puissent être liées (bind) à des éléments de l'interface graphique. Le changement d'une propriété dans l'interface graphique (par exemple la couleur de remplissage) sera alors

automatiquement reporté dans la propriété du modèle de dessin sans que l'on ait besoin d'implémenter un callback.

Les Outils (package tools)

Le package tools contient tous les outils pour réaliser des opérations (sur les figures ou pas) à partir d'évènements souris ([MouseEvent](#)). Parmi ces différentes opérations on peut trouver :

- Le dessin de nouvelles figures dans la zone de dessin (classe `tools.creation.AbstractCreationTool` et héritières) :
 - Pressed → Drag → Release : pour les figures rectangulaires comme les cercles, ellipses et rectangles (classe `tools.creation.RectangularShapeCreationTool`).
- La mise à jour de la position du curseur de la souris dans la zone de dessin (en bas à droite dans l'UI) en utilisant les évènements [MOUSE_MOVED](#) et [MOUSE_EXITED](#) dans la zone de dessin (classe `tools.CursorTool`).
- La mise à jour d'un panneau d'information des figures lorsque le curseur de la souris passe au-dessus d'une figure dans la zone de dessin avec les évènements [MOUSE_ENTERED_TARGET](#) et [MOUSE_EXITED_TARGET](#) (classe `application.panels.InfoPanelController`).
- La sélection ou la désélection de figures directement dans la zone de dessin (classe `tools.SelectionTool`).
- Les transformations géométriques appliquées aux figures dans la zone de dessin (translation, rotation, facteur d'échelle) (classe `tools.TransformTool`).

Interface Graphique avec JavaFX

Nous allons utiliser la séparation entre la définition de l'interface graphique qui sera définie dans le fichier `EditorFrame.fxml` dans le package application et le Contrôleur de cette interface graphique défini dans la classe `Controller` du même package. Le programme principal (classe `Main`) se charge de lire le fichier `EditorFrame.fxml` et d'instancier le contrôleur.

Dans le cadre d'une architecture MVC le fichier `EditorFrame.fxml` définit la Vue, la classe `Drawing` représente le Modèle et la classe `Controller` comme son nom l'indique représente le contrôleur et correspond au moteur de notre application.

Vue

La vue définie dans `EditorFrame.fxml` peut être éditée avec le programme `SceneBuilder` (Voir `SceneBuilder`) et devrait ressembler à ceci :

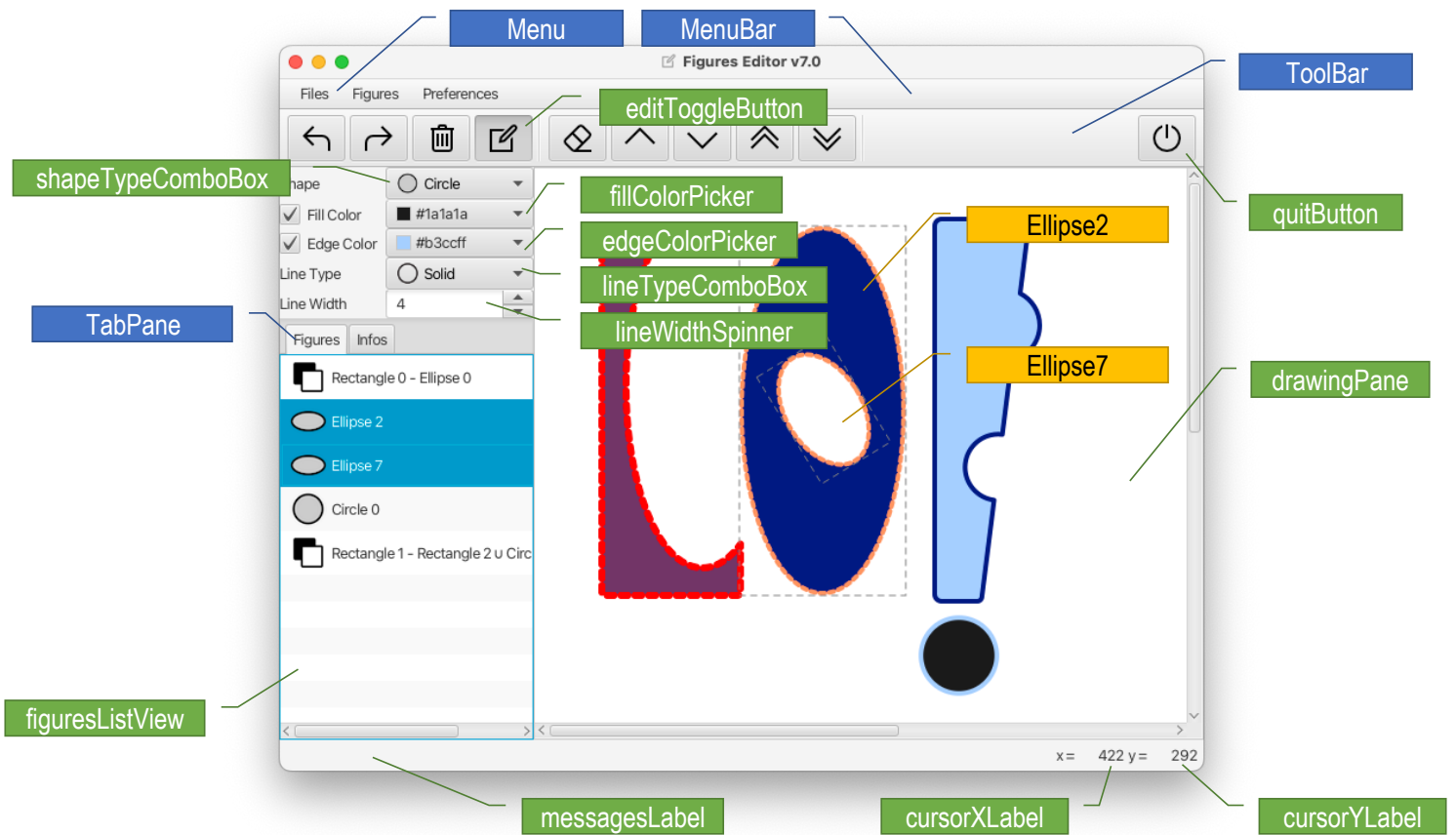
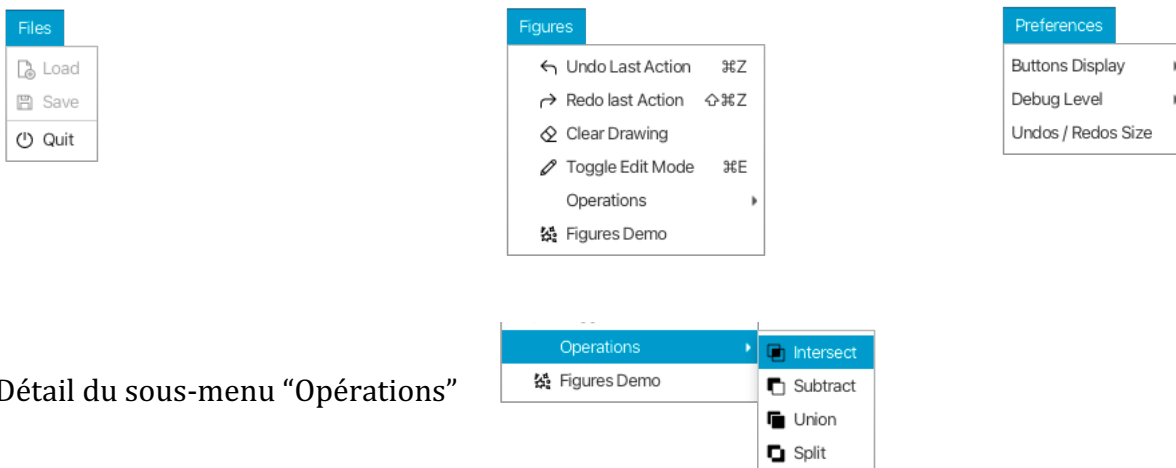


Figure 2 : Éléments de l'interface graphique

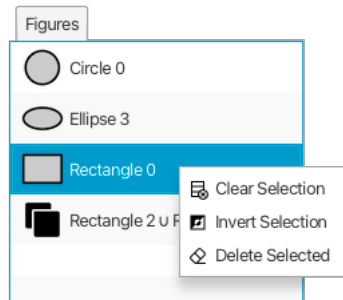
Parmi les autres éléments de la Vue, on peut trouver :

Des menus

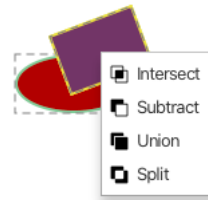


Détail du sous-menu "Opérations"

Des ContextMenus



Dans le figuresListView

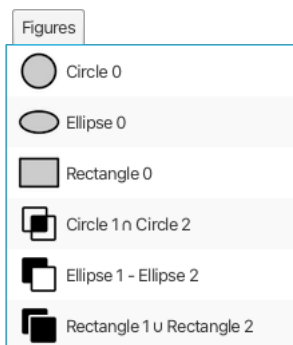


Dans le drawingPane

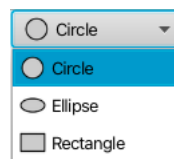
Des widgets customizables

Les `ListView<X>` et `ComboBox<X>` contiennent « effectivement » des éléments de type `X` en JavaFX. Ils sont par ailleurs customisables : On peut créer une Vue customisée (dans un fichier FXML associé à son propre contrôleur) pour les cellules d'un `ListView` ou d'un `ComboBox`.

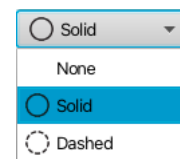
Vous pourrez trouver dans le package `application.cells` un couple de fichiers `FigureCell.fxml` / `FigureCellController.java` permettant de customiser l'affichage des Figures dans le `ListView<Figure>` `figuresListView` (voir Figure 2, page 7). Vous pourrez vous en inspirer pour faire vos propres `CustomCells` pour les types de figures et les types de lignes comme le montre la Figure 3 below.



FigureCell pour afficher des Figures dans un `ListView<Figure>`



FigureTypeCell pour afficher des FigureTypes dans un `ComboBox<FigureType>`



LineTypeCell pour afficher des LineTypes dans un `ComboBox<LineType>`

Figure 3 : Cellules customizées

Contrôleur

La classe `Controller` comme son nom l'indique sert de contrôleur à l'interface graphique définie dans le fichier `EditorFrame.fxml`. Elle contient donc tous les "callbacks" qui permettent de réagir aux différents interactions utilisateurs dans l'interface graphique. Ces callbacks sont implémentés (la plupart du temps) dans les méthodes `onXXXAction(ActionEvent event)` et annotées avec l'annotation `@FXML`. On peut associer ces callbacks à un ou plusieurs éléments de la Vue.

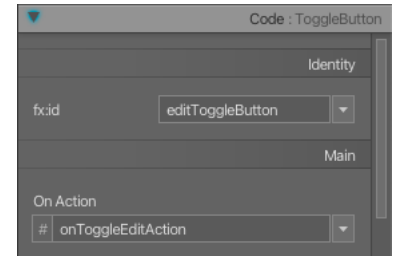
Le contrôleur contient un certain nombre d'attributs (annotés avec l'annotation `@FXML`) qui reflètent certains éléments de l'interface graphique définie dans `EditorFrame.fxml` afin qu'ils puissent être utilisés dans le

contrôleur (typiquement dans les callbacks). Vous devrez veiller à ce que chacun de ces attributs soit bien référencé dans le fichier `EditorFrame.fxml` avec un `fx:id` sans quoi leur utilisation lors du chargement du fichier FXML déclenchera l'erreur "Can't load FXML file ...".

Par exemple, en éditant le fichier `EditorFrame.fxml` dans le logiciel SceneBuilder, et en sélectionnant le `ToggleButton editToggleAction`, on peut lui associer le callback `onEditToggleAction` dans le Controller (Voir Figure 4 ci-dessous) en remplissant le champ "On Action" (le champ `fx:id` n'est pas nécessaire dans ce cas mais peut être utilisé si l'on a besoin d'une référence à ce bouton dans le contrôleur (pour changer son icône par exemple)) :



Figure 4 : Edition dans SceneBuilder



- `fx:id` permet de donner un nom à ce bouton : "editToggleButton". On doit donc retrouver dans la classe Controller un attribut. Exemple :

```
@FXML
private ToggleButton editToggleButton;
```

- `On Action` permet d'associer un callback à ce bouton : "onToggleEditAction". On doit donc retrouver dans la classe Controller une méthode. Exemple :

```
@FXML
public void onToggleEditAction(ActionEvent event)
{
    ...
}
```

Les différentes actions à réaliser dans le Controller ou bien au travers d'outils (voir section [Les Outils \(package tools\)](#)) mis en place par le Controller sont les suivantes :

- File
 -  Save (inutilisé pour l'instant)
 -  Load (inutilisé pour l'instant)
 -  Quit : pour quitter l'application.
-  Création de figures en utilisant des événements souris en utilisant les paramètres suivants :
 - Shape Types
 -  Circle
 -  Ellipse
 -  Rectangle
 - Fill Color en utilisant un `ColorPicker`
 - Applique la couleur de remplissage sélectionnée au modèle.
 - S'il existe des figures sélectionnées, applique ce remplissage aux figures sélectionnées.
 - Stroke Color en utilisant un `ColorPicker`
 - Applique la couleur de contour sélectionnée au modèle.
 - S'il existe des figures sélectionnées, applique cette couleur de contours aux figures sélectionnées.
 - Line Type:

-  Solid
-  Dashed
- None
- Applique le type de ligne des figures au modèle.
- S'il existe des figures sélectionnées, applique ce type de ligne aux figures sélectionnées.
- Line Width allant de 1 to 32 en utilisant un Spinner
 - Applique la largeur de ligne au modèle.
 - S'il existe des figures sélectionnées, applique cette largeur de ligne aux figures sélectionnées.
- Edition
 -  Undo : pour annuler la dernière action.
 -  Redo : pour refaire la dernière action.
 -  Clear : pour effacer le dessin.
 - Figures édition
 -  Move Up : Pour déplacer une figure dans la liste des figures un cran vers l'avant (donc vers le **bas** de la liste).
 -  Move Down : Pour déplacer une figure dans la liste des figures un cran vers l'arrière (donc vers le **haut** de la liste).
 -  Move Top : Pour déplacer une figure dans la liste des figures devant toutes les autres figures (donc en **dernier** dans la liste).
 -  Move Bottom : Pour déplacer une figure dans la liste des figures derrière toutes les autres figures (donc en **premier** dans la liste).
 -  Delete Selected : Pour détruire la ou les figures sélectionnées.
 - Sélection / Désélection des figures en cliquant sur les figures dans la zone de dessin.
 -  Inversion de la sélection des figures.
 - Transformation des figures à la souris (click -> drag -> release)
 - Translate
 - Rotate (avec le modificateur shift)
 - Scale (avec le modificateur ctrl)
 - Création d'une nouvelle figure résultant d'une opération entre deux figures :
 -  Intersection : Intersection des deux figures
 -  Soustraction : Soustraction de la seconde figure à la première
 -  Union : Union des deux figures
 -  Split : Séparation des figures résultant d'opérations en leur composantes respectives.
- Mise à jour de la position du curseur de la zone de dessin dans la barre d'information en bas à droite.
- Mise à jour du panneau d'information (InfoPanel.fxml / InfoPaneController.java) avec les informations d'une figure située sous le curseur de la zone de dessin (Voir Figure 5 ci-dessous qui montre le contenu du panneau d'informations lorsque le curseur se trouve au-dessus de la forme Ellipse0).

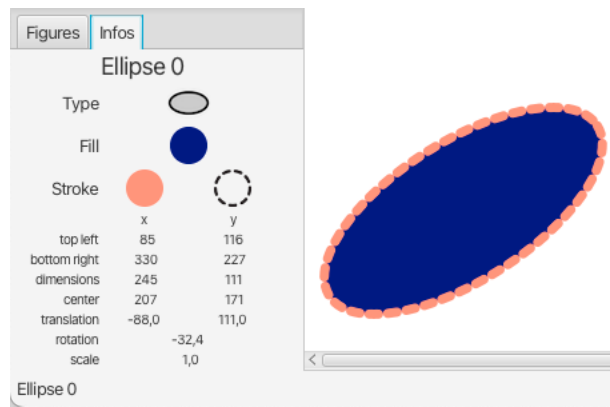


Figure 5 : Panneau d'information

Pour résumer, la classe Controller du package application contient :

- Les attributs annotés @FXML qui reflètent les éléments de la Vue utilisables dans le contrôleur. Et en particulier le Panel de la zone de dessin : `Pane drawingPane`.
- Les attributs propres du contrôleur :
 - Un `Logger` permettant de d'émettre des messages d'information ou de débogage.
 - Une instance du modèle de dessin : `Drawing`.
 - Une instance de `HistoryManager` permettant d'enregistrer les Mementos du modèle de dessin ou de mettre en place un Memento dans le modèle de dessin.
 - Des instances de différents outils :
 - `currentTool` qui contiendra
 - Soit un outil de création de figures adapté au choix du type de figure à créer en mode "Création".
 - Soit l'outil de sélection / désélection de figures en mode "Edition".
 - `transformTool` qui contiendra l'outil permettant de transformer les figures (translation, rotation, facteur d'échelle) en mode "Edition" seulement.
 - `cursorTool` qui met à jour dans la barre d'information en bas à droite de la fenêtre la position du curseur dans la zone de dessin.
 - Un panneau d'information `GridPane infoPanel`; associé à son propre contrôleur `InfoPaneController` qui hérite de `FocusedFigureTool` afin de mettre à jour le panneau d'informations lorsque le curseur de la souris se trouve au-dessus d'une figure dans la zone de dessin.
- Les callbacks qui peuvent être liés aux éléments de la vue : `onXXXAction(ActionEvent event)`.

Travail à réaliser

Ce travail est à réaliser **individuellement**.

De manière générale les classes existantes à compléter contiennent des commentaires :

// TODO XXX class#méthode : actions à effectuer.

- Afin de vous faire découvrir le code du projet, commencez par répondre aux questions du fichier `questions.md` (que vous pourrez trouver dans le projet qui vous est fourni). Vous pourrez déposer votre fichier `questions.md` sur exam.ensiie.fr en utilisant le dépôt `laob-tp4-questions` jusqu'au 20/04/2026 23h59.

- Votre objectif au début du projet sera d'obtenir une première version fonctionnelle de l'application permettant de dessiner des ellipses (la classe `Ellipse` existe déjà ainsi que l'outil de création de formes rectangulaires `RectangularShapeCreationTool`). Pour ce faire vous devrez
 - Commencer par compléter la vue (en éditant le fichier `EditorFrame.fxml` avec `SceneBuilder`). Afin que
 - Chaque attribut annoté `@FXML` dans la classe `Controller` soit aussi indiqué dans le fichier `EditorFrame.fxml`.
 - Chaque callback `onXXXAction(...)` annoté `@FXML` dans la classe `Controller` soit aussi indiqué dans le fichier `EditorFrame.fxml`.
 - Compléter la classe `Figure`.
 - Compléter au moins l'initialisation de la classe `Controller`.
 - Compléter la classe `Drawing` (au moins la méthode `initiateFigure`).
 - Vous pourrez tester la classe `Drawing` avec la classe `test/DrawingTest`.
- Vous pourrez ensuite...
 - Ajouter de nouvelles classes héritant de `Figure` : `Circle` et `Rectangle`.
 - Vous pourrez tester vos classe héritières de la classe `Figure` avec la classe de test `tests/FigureTest` en ayant pris soin de l'éditer pour ajouter vos classes figures.
 - Créer la classe `OperationFigure` permettant de créer des Figures par opérations booléennes sur 2 figures sélectionnées.
 - Vous pourrez créer ces figures avec le `ContextMenu` de la zone de dessin.
 - Lorsque vous aurez complété la classe `Drawing`, vous pourrez commencer à l'utiliser dans les méthodes `onXXXAction` de la classe `Controller`.
 - Ajouter des `CustomCells` pour les `FigureType` et les `LineType`.
 - Compléter l'`HistoryManager` pour pouvoir gérer les Undo/Redos à chaque action ajoutant, supprimant ou modifiant les figures.
 - Il vous faudra aussi utiliser l'`history manager` dans les différents callbacks du `Controller`, Exemples :
 - Pour enregistrer l'état courant avant tout changement :
`// record Drawing model current state before ...`
`historyManager.record();`
 - Pour restaurer le dernier état enregistré par l'`history manager` :
`historyManager.undo();`

Planning

La première séance dédiée à ce mini-projet aura lieu lundi 20/04/2026 et la dernière séance lundi 11/05/2026 pour un total de 6 séances d'1h45. Il vous est fortement recommandé de travailler chez vous en dehors des séances encadrées sur ce projet. Vous pourrez alors profiter des séances encadrées pour soumettre les problèmes que vous aurez rencontrés à votre chargé de TD.

Vous pourrez déposer une archive de votre projet au plus tard vendredi 22/05/2026 23:59 sur exam.ensiie.fr en utilisant le dépôt `laob-tp4`.

Mise en place et utilisation de JavaFX

Un tutoriel complet d'installation et d'utilisation de JavaFX dans les différents IDEs tels que Eclipse, IntelliJ, NetBeans et Visual Studio Code est disponible à l'adresse : <https://openjfx.io/openjfx-docs/>.

Ce qui suit ne fait que refléter ce tutoriel en utilisant les machines de l'école.

SceneBuilder

SceneBuilder est l'application qui vous permet d'éditer les fichiers fxml qui contiennent des description XML d'interfaces graphiques utilisables par JavaFX.

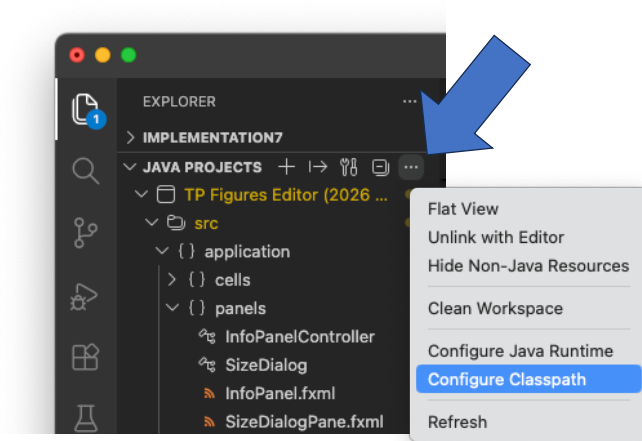
Vous pouvez télécharger l'application SceneBuilder à l'adresse suivante :

<https://gluonhq.com/products/scene-builder/>. Elle contient sa propre version de JavaFX.

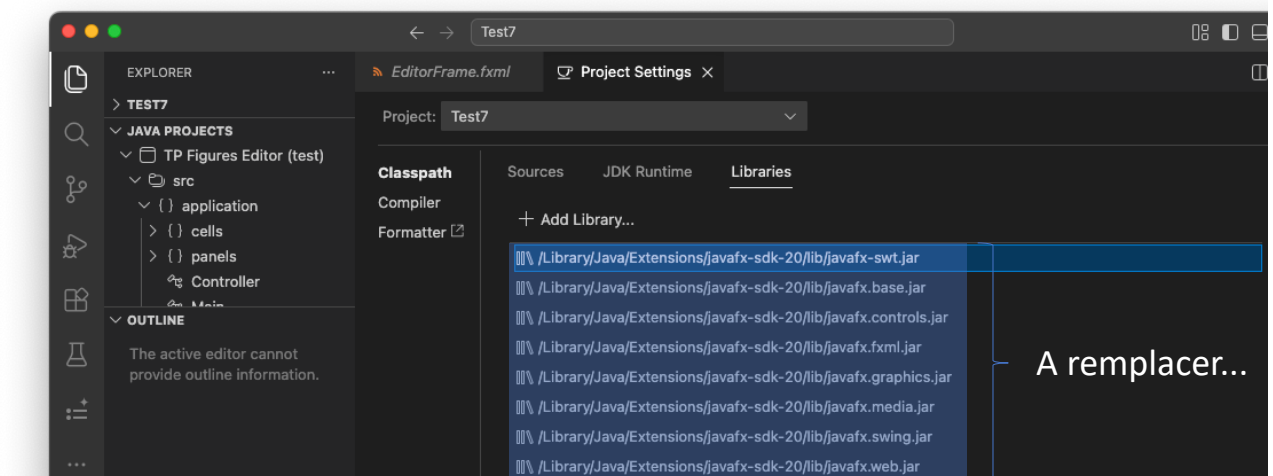
A l'école, l'application SceneBuilder est installée dans /opt/scenebuilder/bin/ et s'appelle SceneBuilder.

Mise en place de JavaFX dans VSCode

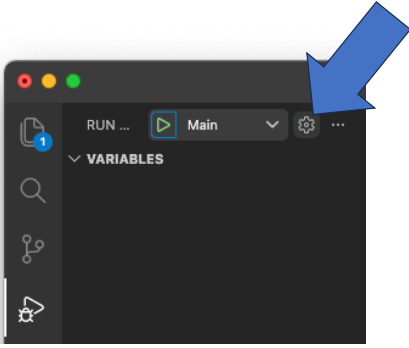
- Pour utiliser JavaFX dans VSCode, vous aurez besoin d'une version de VSCode équipée du plugin [JavaFX Support](#) de Shrey Pandya. A priori, la version de VSCode installée par Mr Mouilleron dans /pub/FISE_IPFL12/code contient déjà ce plugin.
- Une fois ce plugin installé dans VSCode il faut configurer l'accès à SceneBuilder afin que vous puissiez lancer SceneBuilder depuis VSCode : Ouvrez la palette (Shift-Ctrl-P) et tapez "Configure SceneBuilder Path" pour lui indiquer où se trouve SceneBuilder. Par la suite, vous pourrez sélectionner un fichier .fxml dans votre projet et à nouveau utiliser la palette "Open in SceneBuilder".
- Ensuite, il faut indiquer à VSCode où trouver JavaFX. Pour ce faire,
 - cliquez sur les "..." dans le bloc "Java Project" :



- Puis il vous faudra remplacer (un à un) les .jar de JavaFX par les fichiers .jar que vous pourrez trouver dans /pub/FISE_LA0B12/javafx-sdk-20/lib :



- Pour pouvoir lancer le programme principal, il vous faudra customiser la config de lancement du programme principal. Éditez le fichier `.vscode/launch.json` en cliquant sur la roue dentée :



- Puis changez le champs "vmArgs" afin qu'il reflète votre configuration :

```
{
  "type": "java",
  "name": "Main",
  "request": "launch",
  "mainClass": "application.Main",
  "vmArgs": "--module-path=/pub/FISE_LA0B12/javafx-sdk-20/lib --add-modules=javafx.controls,javafx.fxml"
}
```

- Et enfin, pour que les tests (qui utilisent aussi JavaFX) se déroulent sans accroc il vous faudra éditer le fichier `.vscode/settings.json` pour ajouter les éléments suivants :

```
"java.test.config": {
  "vmArgs": [
    "--module-path=/pub/FISE_LA0B12/javafx-sdk-20/lib",
    "--add-modules=javafx.controls,javafx.fxml,javafx.graphics,javafx.swing",
    "--enable-native-access=javafx.graphics"
  ]
},
```

Mise en place de JavaFX dans Eclipse

- Pour utiliser JavaFX dans Eclipse, vous aurez besoin d'une version d'Eclipse équipée du plugin [E\(fx\)clipse](#). A priori, la version d'Eclipse installée à l'école contient déjà ce plugin.
- Une fois cette version d'Eclipse lancée, il vous faudra la configurer (Preferences → JavaFX) pour lui indiquer où se trouvent
 - JavaFX : qui se trouve dans le répertoire `/pub/FISE_LA0B12/javafx-sdk-20/`.
 - SceneBuilder qui vous permettra d'ouvrir les fichiers FXML pour les éditer graphiquement. Le programme SceneBuilder se trouve dans le répertoire `/opt/scenbuilder/bin/SceneBuilder`.
 - Ceci vous permettra de créer des projets JavaFX mais pas d'importer le projet que l'on vous fournit.
- Pour pouvoir importer et faire tourner le projet qu'on vous fournit vous allez avoir besoin :
 - De définir une « User Library » dans Eclipse correspondant à JavaFX : Eclipse → Window → Preferences → Java → Build Path → User Libraries → New et appelez la « JavaFX », puis éditez cette nouvelle librairie avec « Add External JARs... » et ajoutez tous les fichiers `.jar` contenu dans `/pub/FISE_LA0B12/javafx-sdk-20/lib`.
 - Dans le projet « Figure Editor » faites un clic droit sur la racine du projet et éditez ses « Properties » → Java Build Path et ajoutez la « User Library » que vous venez de créer au ClassPath en remplaçant au besoin celle qui s'y trouvait déjà. Votre projet est maintenant compilable, il nous reste à effectuer une dernière étape pour que vous puissiez lancer l'exécution du `Main.java` du package `application`.

- Si vous tentez de lancer Main.java : Run As → Java Application vous allez avoir une erreur car il faut préciser à java où trouver les modules JavaFX. Pour ce faire, au lieu de Java Application, sélectionnez Run As → Run Configurations.... Double cliquez sur Java Application pour créer une nouvelle configuration que vous appellerez « Figure Editor » pour la retrouver facilement par la suite.
- Dans cette nouvelle configuration, allez à l'onglet « Arguments » et collez les arguments suivants dans le champ « VM Arguments » : `-module-path ${PATH_TO_FX} -add-modules javafx.controls,javafx.fxml,javafx.graphics`
- La variable `${PATH_TO_FX}` doit être définie en cliquant sur le bouton “Variables” et en ajoutant une variable `PATH_TO_FX` contenant le chemin vers les librairies JavaFX : `/pub/FISE_LAOB12/javafx-sdk-20/lib`
- Cliquez sur le bouton « run » en bas à droite et votre application devrait se lancer. Pour les lancements suivants, l'utilisation du bouton run dans la barre d'outils en haut d'Eclipse devrait suffire.